

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 10

TRANSACTIONS, PROGRAMMABILITY AND SECURITY

1. Aim

To implement transactions using COMMIT, ROLLBACK and SAVEPOINT, to develop stored procedures, functions and triggers, to implement role-based access control in MySQL, and to perform basic NoSQL operations in MongoDB.

2. Problem Statement

Extend *college_db* with an online fee-payment facility and secure access:

1. Each student has a wallet. Paying a fee must debit the wallet, credit the college account and generate a receipt as one atomic transaction; demonstrate COMMIT, ROLLBACK, SAVEPOINT and failure inside a transaction.
2. Write a function that converts marks to a grade, a procedure *pay_fee* that performs a payment safely and reports insufficient balance, and a procedure that returns the number of students and average marks of a department through OUT parameters.
3. Write triggers that always compute the grade from marks and keep an audit trail of every change of marks with the user who made it.
4. Create a faculty role that can read all data and update only marks, and a student role that can only read results through a view; create one user for each role and test them.
5. Store student documents with embedded courses in MongoDB and perform CRUD operations, an aggregation and indexing.

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To understand ACID properties and control transactions using COMMIT, ROLLBACK and SAVEPOINT.
2. To write stored functions and stored procedures with parameters and error handling.
3. To automate actions using BEFORE and AFTER triggers.
4. To implement role-based access control with CREATE ROLE, GRANT and REVOKE.
5. To perform basic CRUD and aggregation operations in a NoSQL document database (MongoDB).

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files
5	MongoDB Community Server 7.0 with MongoDB Shell (mongosh)	Document (NoSQL) database for Q16-Q17

5. Theory

5.1 Transactions and ACID Properties

A **transaction** is a sequence of operations performed as a single logical unit of work, e.g. debiting a student's wallet and crediting the college account. The DBMS guarantees the **ACID** properties:

Property	Meaning	Ensured by (InnoDB)
Atomicity	All operations succeed or none take effect	Undo log, ROLLBACK
Consistency	A transaction moves the database from one valid state to another	Constraints, triggers, application logic
Isolation	Concurrent transactions do not see each other's uncommitted changes	Locks and MVCC; default level REPEATABLE READ
Durability	Committed changes survive crashes	Redo log (write-ahead logging)

MySQL runs with `autocommit = 1`, so each statement is a transaction by itself unless `START TRANSACTION` is used. `COMMIT` makes changes permanent, `ROLLBACK` undoes them, and `SAVEPOINT name / ROLLBACK TO SAVEPOINT` undo only part of a transaction. DDL statements cause an implicit commit. A statement that fails inside a transaction is undone by itself, but earlier statements remain until `ROLLBACK` is issued.

5.2 Stored Programs

- **Stored procedure** - a named block of SQL statements stored in the server and executed with `CALL`. It can have IN, OUT and INOUT parameters, local variables (`DECLARE`), control flow (`IF`, `CASE`, `LOOP`, `WHILE`), cursors and error handlers (`DECLARE EXIT | CONTINUE HANDLER FOR ...`). `SIGNAL SQLSTATE '45000'` raises a user-defined error.
- **Stored function** - returns a single value with `RETURN` and can be used inside SQL expressions. With binary logging enabled MySQL requires it to be declared `DETERMINISTIC`, `NO SQL` or `READS SQL DATA`.
- **Trigger** - a stored program executed automatically `BEFORE` or `AFTER` an `INSERT`, `UPDATE` or `DELETE` on a table, `FOR EACH ROW`. `OLD` and `NEW` refer to the row values before and after the change; `BEFORE` triggers may modify `NEW`. Triggers are used for derived values, validation and auditing.

The mysql client treats ';' as the end of a statement, so the delimiter is temporarily changed (DELIMITER \$\$) while creating a stored program whose body contains ';'. **DBMS differences:** SQL Server uses T-SQL (CREATE PROCEDURE ... AS BEGIN ... END, BEGIN TRAN), Oracle uses PL/SQL and PostgreSQL uses PL/pgSQL with trigger functions.

5.3 Security and Role-Based Access Control (RBAC)

Access control follows the **principle of least privilege**: each user receives only the privileges needed for his/her work. Privileges (SELECT, INSERT, UPDATE, DELETE, EXECUTE, ...) can be granted at global, database, table, column and routine level. In **RBAC**, privileges are granted to **roles** (MySQL 8.0+) and roles are granted to users, which simplifies administration when many users have the same job.

```
CREATE ROLE 'role';
CREATE USER 'u'@'host' IDENTIFIED BY 'password';
GRANT 'role' TO 'u'@'host';
SHOW GRANTS FOR 'u'@'host' USING 'role';
REVOKE privilege ON db.object FROM 'role';
```

Views (Practical 9) are combined with GRANT to expose only selected columns and rows to a role.

5.4 NoSQL Databases and MongoDB

NoSQL databases relax the fixed relational schema to support flexible, large-scale and distributed data. Main types are key-value (Redis), document (MongoDB), column-family (Cassandra) and graph (Neo4j) stores. MySQL itself is relational; its JSON data type and Document Store (X DevAPI) add document features, but the standard NoSQL demonstration in this practical uses **MongoDB**, a document database that stores BSON (binary JSON) documents in collections.

Relational (MySQL)	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field (may hold arrays and embedded documents)
Primary key	_id field
JOIN	Embedding or \$lookup
INSERT / SELECT / UPDATE / DELETE	insertOne/insertMany, find, updateOne/updateMany, deleteOne/deleteMany
GROUP BY	aggregate() pipeline with \$group

6. Procedure / Algorithm

1. Create and populate *college_db* (Q1, Q2) and create the *wallet*, *college_account* and *fee_payment* tables (Q3).
2. Execute a fee-payment transaction and COMMIT it; execute another and ROLLBACK it; use a SAVEPOINT for partial rollback; observe the behaviour when a statement inside a transaction fails.
3. Create the stored function *get_grade* and use it in SELECT.
4. Create the stored procedure *pay_fee* with a transaction, an EXIT HANDLER and SIGNAL; call it for a valid and an invalid payment.
5. Create the procedure *dept_stats* with OUT parameters and call it using session variables.
6. Create BEFORE INSERT/UPDATE triggers that compute the grade and an AFTER UPDATE trigger that writes an audit record; test them.
7. Create roles *faculty_role* and *student_role*, grant privileges, create users, assign roles and display the grants.
8. Log in as each user and verify allowed and denied operations; revoke a privilege.
9. Start the MongoDB server, open mongosh and perform insert, find, update, delete, aggregation and index operations on a students collection.

7. SQL Queries / Program

Q1: Setup: create college_db and its tables (from Practical 2)

```

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id      INT          NOT NULL,
    dept_code    VARCHAR(10)  NOT NULL,
    dept_name    VARCHAR(40)  NOT NULL,
    building     VARCHAR(20),
    budget       DECIMAL(12,2) DEFAULT 1000000.00,
    CONSTRAINT pk_department PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_code  UNIQUE (dept_code),
    CONSTRAINT chk_budget    CHECK (budget > 0)
);

CREATE TABLE instructor (
    instr_id      INT          NOT NULL,
    instr_name    VARCHAR(40)  NOT NULL,
    designation   VARCHAR(20)  NOT NULL,
    dept_id       INT,
    salary        DECIMAL(10,2) NOT NULL,
    email         VARCHAR(40)  NOT NULL,
    mentor_id     INT,
    CONSTRAINT pk_instructor PRIMARY KEY (instr_id),
    CONSTRAINT uq_instr_email UNIQUE (email),
    CONSTRAINT chk_salary   CHECK (salary BETWEEN 30000 AND 300000),
    CONSTRAINT fk_instr_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_instr_mentor FOREIGN KEY (mentor_id)
        REFERENCES instructor (instr_id) ON DELETE SET NULL
);

```

```

CREATE TABLE student (
    student_id    INT            NOT NULL AUTO_INCREMENT,
    roll_no       VARCHAR(10)    NOT NULL,
    student_name  VARCHAR(40)    NOT NULL,
    gender        CHAR(1)        NOT NULL,
    dob           DATE           NOT NULL,
    dept_id       INT            NOT NULL,
    city          VARCHAR(30)    DEFAULT 'Pune',
    email         VARCHAR(40)    NOT NULL,
    CONSTRAINT pk_student      PRIMARY KEY (student_id),
    CONSTRAINT uq_roll_no      UNIQUE (roll_no),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT chk_gender      CHECK (gender IN ('M', 'F', 'O')),
    CONSTRAINT chk_dob         CHECK (dob >= '1995-01-01'),
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON UPDATE CASCADE
);

CREATE TABLE student_phone (
    student_id    INT            NOT NULL,
    phone         CHAR(10)       NOT NULL,
    CONSTRAINT pk_student_phone PRIMARY KEY (student_id, phone),
    CONSTRAINT chk_phone        CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_phone_student FOREIGN KEY (student_id)
        REFERENCES student (student_id) ON DELETE CASCADE
);

CREATE TABLE course (
    course_id     CHAR(5)        NOT NULL,
    title         VARCHAR(40)    NOT NULL,
    dept_id       INT            NOT NULL,
    credits       TINYINT        NOT NULL,
    instr_id      INT,
    prereq_id     CHAR(5),
    CONSTRAINT pk_course      PRIMARY KEY (course_id),
    CONSTRAINT uq_course_title UNIQUE (title),
    CONSTRAINT chk_credits    CHECK (credits BETWEEN 1 AND 5),
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON UPDATE CASCADE,
    CONSTRAINT fk_course_instr FOREIGN KEY (instr_id)
        REFERENCES instructor (instr_id) ON DELETE SET NULL,
    CONSTRAINT fk_course_prereq FOREIGN KEY (prereq_id)
        REFERENCES course (course_id)
);

CREATE TABLE enrollment (
    student_id    INT            NOT NULL,
    course_id     CHAR(5)        NOT NULL,
    semester      TINYINT        NOT NULL,
    marks         INT,
    grade         CHAR(2),
    CONSTRAINT pk_enrollment  PRIMARY KEY (student_id, course_id),
    CONSTRAINT chk_semester   CHECK (semester BETWEEN 1 AND 8),
    CONSTRAINT chk_marks      CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
        REFERENCES student (student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_course  FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

This setup script recreates the common *college_db* case-study database (schema of Practical 2 and data of Practical 3) so that this practical can be executed independently on a fresh MySQL server.

Q2: Setup: insert the sample data (from Practical 3)

```

INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES
  (10, 'CSE', 'Computer Engineering', 'A Block', 2500000),
  (20, 'AIML', 'AI and Machine Learning', 'B Block', 3000000),
  (30, 'ECE', 'Electronics and Telecomm', 'C Block', 1800000),
  (40, 'MECH', 'Mechanical Engineering', 'D Block', 1500000),
  (50, 'CIVIL', 'Civil Engineering', 'D Block', 1200000);

INSERT INTO instructor VALUES
  (101, 'Anil Kulkarni', 'Professor', 10, 185000, 'anil.k@example.com', NULL),
  (102, 'Meera Joshi', 'Professor', 20, 172000, 'meera.j@example.com', 101),
  (103, 'Rahul Patil', 'Asst. Prof', 10, 82000, 'rahul.p@example.com', 101),
  (104, 'Sneha Deshmukh', 'Assoc. Prof', 20, 115000, 'sneha.d@example.com', 102),
  (105, 'Vikram Shinde', 'Professor', 30, 168000, 'vikram.s@example.com', 101),
  (106, 'Pooja Kale', 'Asst. Prof', 20, 78000, 'pooja.k@example.com', 104),
  (107, 'Nitin Pawar', 'Asst. Prof', 50, 76000, 'nitin.p@example.com', 105),
  (108, 'Kavita More', 'Assoc. Prof', 40, 112000, 'kavita.m@example.com', 105);

INSERT INTO student
  (roll_no, student_name, gender, dob, dept_id, city, email) VALUES
  ('S24CS001', 'Aarav Sharma', 'M', '2005-03-14', 10, 'Pune', 'aarav@example.com'),
  ('S24CS002', 'Isha Gupta', 'F', '2005-07-22', 10, 'Mumbai', 'isha@example.com'),
  ('S24AI001', 'Rohan Mehta', 'M', '2005-01-09', 20, 'Pune', 'rohan@example.com'),
  ('S24AI002', 'Priya Nair', 'F', '2004-11-30', 20, 'Nashik', 'priya@example.com'),
  ('S24AI003', 'Kunal Verma', 'M', '2005-05-18', 20, 'Pune', 'kunal@example.com'),
  ('S24EC001', 'Sneha Iyer', 'F', '2005-09-02', 30, 'Nagpur', 'sneha@example.com'),
  ('S24EC002', 'Aditya Rao', 'M', '2004-12-25', 30, 'Mumbai', 'aditya@example.com'),
  ('S24ME001', 'Neha Kulkarni', 'F', '2005-02-11', 40, 'Pune', 'neha@example.com'),
  ('S24CS003', 'Yash Jadhav', 'M', '2005-08-05', 10, NULL, 'yash@example.com'),
  ('S24AI004', 'Ananya Desai', 'F', '2005-04-27', 20, 'Mumbai', 'ananya@example.com');

INSERT INTO student_phone VALUES
  (1, '9876500011'), (1, '9876500012'), (2, '9876500021'), (3, '9876500031'),
  (4, '9876500041'), (4, '9876500042'), (6, '9876500061'), (9, '9876500091');

INSERT INTO course VALUES
  ('CS101', 'Programming in C', 10, 4, 103, NULL),
  ('CS201', 'Data Structures', 10, 4, 103, 'CS101'),
  ('CS301', 'Database Management Systems', 10, 3, 101, 'CS201'),
  ('CS401', 'Advanced Databases', 10, 3, 101, 'CS301'),
  ('AI201', 'Python for Data Science', 20, 3, 104, 'CS101'),
  ('AI301', 'Machine Learning', 20, 4, 102, 'AI201'),
  ('AI401', 'Deep Learning', 20, 3, 106, 'AI301'),
  ('EC201', 'Digital Electronics', 30, 3, 105, NULL),
  ('EC301', 'Signals and Systems', 30, 4, NULL, 'EC201'),
  ('ME201', 'Engineering Mechanics', 40, 3, 108, NULL);

INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES
  (1, 'CS101', 3, 78, 'A'), (1, 'CS201', 3, 85, 'A+'), (1, 'CS301', 4, 91, 'O'),
  (1, 'AI201', 3, 72, 'A'), (2, 'CS101', 3, 88, 'A+'), (2, 'CS201', 3, 67, 'B+'),
  (2, 'CS301', 4, 74, 'A'), (3, 'CS101', 3, 65, 'B+'), (3, 'AI201', 3, 81, 'A+'),
  (3, 'AI301', 4, 89, 'A+'), (3, 'CS301', 4, 70, 'A'), (4, 'AI201', 3, 92, 'O'),
  (4, 'AI301', 4, 76, 'A'), (4, 'AI401', 4, 84, 'A+'), (5, 'AI201', 3, 58, 'B'),
  (5, 'AI301', 4, 63, 'B+'), (6, 'EC201', 3, 79, 'A'), (6, 'CS101', 3, 55, 'B'),
  (6, 'EC301', 4, 82, 'A+'), (7, 'EC201', 3, 45, 'C'), (7, 'EC301', 4, 61, 'B+'),
  (8, 'CS101', 3, 78, 'A'), (9, 'CS201', 3, 90, 'O'), (9, 'CS301', 4, 95, 'O'),
  (9, 'CS401', 4, 88, 'A+');

```

Q3: Setup: wallet, college account and fee-payment tables

```

CREATE TABLE wallet (
    student_id INT PRIMARY KEY,
    balance DECIMAL(10,2) NOT NULL,
    CONSTRAINT chk_balance CHECK (balance >= 0),
    FOREIGN KEY (student_id) REFERENCES student (student_id)
);
CREATE TABLE college_account (
    acc_id INT PRIMARY KEY,
    balance DECIMAL(12,2) NOT NULL
);
CREATE TABLE fee_payment (
    receipt_no INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),
    FOREIGN KEY (student_id) REFERENCES student (student_id)
);
INSERT INTO wallet VALUES (1, 60000), (2, 45000), (3, 15000), (4, 80000);
INSERT INTO college_account VALUES (1, 500000);

```

Q4: Transaction 1: fee payment completed with COMMIT

```

SELECT @@autocommit;
START TRANSACTION;
UPDATE wallet SET balance = balance - 40000 WHERE student_id = 1;
UPDATE college_account SET balance = balance + 40000 WHERE acc_id = 1;
INSERT INTO fee_payment (student_id, amount) VALUES (1, 40000);
COMMIT;
SELECT w.student_id, w.balance AS wallet_balance, a.balance AS college_balance
FROM wallet w, college_account a
WHERE w.student_id = 1 AND a.acc_id = 1;

```

Q5: Transaction 2: changes cancelled with ROLLBACK

```

START TRANSACTION;
UPDATE wallet SET balance = balance - 40000 WHERE student_id = 2;
UPDATE college_account SET balance = balance + 40000 WHERE acc_id = 1;
SELECT student_id, balance FROM wallet WHERE student_id = 2;
ROLLBACK;
SELECT student_id, balance FROM wallet WHERE student_id = 2;
SELECT balance AS college_balance FROM college_account WHERE acc_id = 1;

```

Q6: Transaction 3: partial rollback using SAVEPOINT

```

START TRANSACTION;
UPDATE wallet SET balance = balance - 5000 WHERE student_id = 4;
SAVEPOINT after_exam_fee;
UPDATE wallet SET balance = balance - 30000 WHERE student_id = 4;
SELECT student_id, balance FROM wallet WHERE student_id = 4;
ROLLBACK TO SAVEPOINT after_exam_fee;
COMMIT;
SELECT student_id, balance FROM wallet WHERE student_id = 4;

```

Q7: Transaction 4: an error inside a transaction (atomicity)

```

START TRANSACTION;
UPDATE college_account SET balance = balance + 40000 WHERE acc_id = 1;
UPDATE wallet SET balance = balance - 40000 WHERE student_id = 3;
ROLLBACK;
SELECT (SELECT balance FROM wallet WHERE student_id = 3) AS wallet_3,
       (SELECT balance FROM college_account WHERE acc_id = 1) AS college_balance;

```

Student 3 has only 15000 in the wallet, so the second UPDATE violates chk_balance. MySQL rejects only that statement; the application must then ROLLBACK so that the credit already made to the college account is also undone.

Q8: Stored function: grade from marks

```
DELIMITER $$
CREATE FUNCTION get_grade(p_marks INT)
RETURNS CHAR(2)
DETERMINISTIC
BEGIN
    RETURN CASE
        WHEN p_marks IS NULL THEN NULL
        WHEN p_marks >= 90 THEN 'O'   WHEN p_marks >= 80 THEN 'A+'
        WHEN p_marks >= 70 THEN 'A'   WHEN p_marks >= 60 THEN 'B+'
        WHEN p_marks >= 50 THEN 'B'   WHEN p_marks >= 40 THEN 'C'
        ELSE 'F'
    END;
END$$
DELIMITER ;

SELECT get_grade(95) AS g95, get_grade(83) AS g83,
       get_grade(47) AS g47, get_grade(12) AS g12;
SELECT student_id, course_id, marks, grade, get_grade(marks) AS computed
FROM   enrollment WHERE course_id = 'CS301';
```

Q9: Stored procedure with transaction and error handling: pay_fee

```
DELIMITER $$
CREATE PROCEDURE pay_fee(IN p_student INT, IN p_amount DECIMAL(10,2))
BEGIN
    DECLARE v_balance DECIMAL(10,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
    START TRANSACTION;
    SELECT balance INTO v_balance FROM wallet
    WHERE student_id = p_student FOR UPDATE;
    IF v_balance IS NULL OR v_balance < p_amount THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient wallet balance';
    END IF;
    UPDATE wallet SET balance = balance - p_amount WHERE student_id = p_student;
    UPDATE college_account SET balance = balance + p_amount WHERE acc_id = 1;
    INSERT INTO fee_payment (student_id, amount) VALUES (p_student, p_amount);
    COMMIT;
    SELECT CONCAT('Receipt ', LAST_INSERT_ID(), ': fee of ', p_amount,
                  ' paid by student ', p_student) AS message;
END$$
DELIMITER ;

CALL pay_fee(2, 40000);
CALL pay_fee(3, 40000);
SELECT * FROM fee_payment;
SELECT * FROM wallet;
```

Q10: Stored procedure with OUT parameters

```

DELIMITER $$
CREATE PROCEDURE dept_stats(IN p_dept_code VARCHAR(10),
                           OUT p_students INT, OUT p_avg DECIMAL(5,2))
BEGIN
    SELECT COUNT(DISTINCT s.student_id), AVG(e.marks)
    INTO    p_students, p_avg
    FROM    student s
    JOIN    department d ON d.dept_id = s.dept_id
    LEFT JOIN enrollment e ON e.student_id = s.student_id
    WHERE   d.dept_code = p_dept_code;
END$$
DELIMITER ;

CALL dept_stats('AIML', @n, @avg);
SHOW WARNINGS;
SELECT @n AS aiml_students, @avg AS aiml_avg_marks;

```

The warning is a note that the exact average (75.333...) was rounded to fit the declared OUT parameter type DECIMAL(5,2).

Q11: Triggers: automatic grade and audit trail of marks changes

```

CREATE TABLE marks_audit (
    audit_id      INT AUTO_INCREMENT PRIMARY KEY,
    student_id    INT, course_id CHAR(5),
    old_marks     INT, new_marks INT,
    changed_by    VARCHAR(60)
);

CREATE TRIGGER trg_enrollment_bi BEFORE INSERT ON enrollment
FOR EACH ROW SET NEW.grade = get_grade(NEW.marks);

CREATE TRIGGER trg_enrollment_bu BEFORE UPDATE ON enrollment
FOR EACH ROW SET NEW.grade = get_grade(NEW.marks);

DELIMITER $$
CREATE TRIGGER trg_enrollment_au AFTER UPDATE ON enrollment
FOR EACH ROW
BEGIN
    IF NOT (OLD.marks <=> NEW.marks) THEN
        INSERT INTO marks_audit
            (student_id, course_id, old_marks, new_marks, changed_by)
        VALUES (OLD.student_id, OLD.course_id, OLD.marks, NEW.marks, USER());
    END IF;
END$$
DELIMITER ;

INSERT INTO enrollment (student_id, course_id, semester, marks)
VALUES (10, 'AI201', 3, 86);
UPDATE enrollment SET marks = 52 WHERE student_id = 7 AND course_id = 'EC201';
SELECT * FROM enrollment
WHERE (student_id, course_id) IN ((10, 'AI201'), (7, 'EC201'));
SELECT * FROM marks_audit;
SELECT trigger_name, action_timing, event_manipulation, event_object_table
FROM    information_schema.triggers
WHERE   trigger_schema = 'college_db';

```

The grade supplied by the user is ignored: the BEFORE triggers always compute it from the marks, so the two columns can never become inconsistent.

Q12: Role-based access control: roles, privileges and users

```

DROP USER IF EXISTS 'prof_rahul'@'localhost', 'stu_aarav'@'localhost';
DROP ROLE IF EXISTS 'faculty_role', 'student_role';

CREATE VIEW v_course_results AS
SELECT s.roll_no, e.course_id, e.marks, e.grade
FROM enrollment e JOIN student s ON s.student_id = e.student_id;

CREATE ROLE 'faculty_role', 'student_role';
GRANT SELECT ON college_db.* TO 'faculty_role';
GRANT UPDATE (marks) ON college_db.enrollment TO 'faculty_role';
GRANT EXECUTE ON PROCEDURE college_db.dept_stats TO 'faculty_role';
GRANT SELECT ON college_db.v_course_results TO 'student_role';

CREATE USER 'prof_rahul'@'localhost' IDENTIFIED BY 'Faculty@123';
CREATE USER 'stu_aarav'@'localhost' IDENTIFIED BY 'Student@123';
GRANT 'faculty_role' TO 'prof_rahul'@'localhost';
GRANT 'student_role' TO 'stu_aarav'@'localhost';
SET DEFAULT ROLE ALL TO 'prof_rahul'@'localhost', 'stu_aarav'@'localhost';

SHOW GRANTS FOR 'faculty_role';
SHOW GRANTS FOR 'stu_aarav'@'localhost' USING 'student_role';

```

Q13: Testing the faculty account

Executed in a separate session logged in as prof_rahul@localhost.

```

-- Login: mysql -u prof_rahul -p college_db
SELECT CURRENT_USER(), CURRENT_ROLE();
SELECT * FROM v_course_results WHERE course_id = 'CS101';
UPDATE enrollment SET marks = 80 WHERE student_id = 8 AND course_id = 'CS101';
UPDATE enrollment SET grade = 'O' WHERE student_id = 8 AND course_id = 'CS101';
DELETE FROM student WHERE student_id = 8;
CALL dept_stats('CSE', @n, @avg);
SELECT @n, @avg;

```

Q14: Testing the student account

Executed in a separate session logged in as stu_aarav@localhost.

```

-- Login: mysql -u stu_aarav -p college_db
SELECT * FROM v_course_results WHERE roll_no = 'S24CS001';
SELECT * FROM wallet;
UPDATE v_course_results SET marks = 100 WHERE roll_no = 'S24CS001';

```

Q15: Audit trail after the faculty update; revoking a privilege

```

SELECT * FROM marks_audit;
REVOKE UPDATE (marks) ON college_db.enrollment FROM 'faculty_role';
SHOW GRANTS FOR 'faculty_role';

```

Q16: Basic NoSQL operations in MongoDB (mongosh): create, read, update, delete

```

use college_nosql

db.students.drop()

db.students.insertMany([
  { _id: 1, roll_no: "S24CS001", name: "Aarav Sharma", dept: "CSE", city: "Pune",
    skills: ["C", "SQL"],
    courses: [ { code: "CS101", marks: 78 }, { code: "CS301", marks: 91 } ] },
  { _id: 2, roll_no: "S24CS002", name: "Isha Gupta", dept: "CSE", city: "Mumbai",
    skills: ["Java"],
    courses: [ { code: "CS101", marks: 88 }, { code: "CS301", marks: 74 } ] },
])

```

```

{ _id: 3, roll_no: "S24AI001", name: "Rohan Mehta", dept: "AIML", city: "Pune",
  skills: ["Python"],
  courses: [ { code: "AI201", marks: 81 }, { code: "AI301", marks: 89 } ] },
{ _id: 4, roll_no: "S24AI002", name: "Priya Nair", dept: "AIML", city: "Nashik",
  skills: ["Python", "R"],
  courses: [ { code: "AI201", marks: 92 }, { code: "AI401", marks: 84 } ] },
{ _id: 5, roll_no: "S24EC001", name: "Sneha Iyer", dept: "ECE", city: "Nagpur",
  courses: [ { code: "EC201", marks: 79 } ] }
])

db.students.find({ city: "Pune" }, { _id: 0, roll_no: 1, name: 1, dept: 1 })

db.students.find({ "courses.marks": { $gte: 90 } }, { _id: 0, name: 1 })

db.students.countDocuments({ dept: "AIML" })

db.students.updateOne({ _id: 3 },
  { $set: { city: "Mumbai" }, $push: { skills: "TensorFlow" } })

db.students.updateMany({ dept: "AIML" }, { $set: { year: "SY" } })

db.students.findOne({ _id: 3 })

db.students.deleteOne({ _id: 5 })

db.students.countDocuments()

```

Executed in the MongoDB Shell (mongosh) connected to a MongoDB 7.0 server.

Q17: MongoDB aggregation pipeline, unique index and query plan

```

use college_nosql

db.students.aggregate([
  { $unwind: "$courses" },
  { $group: { _id: "$dept", enrollments: { $sum: 1 },
    avg_marks: { $avg: "$courses.marks" } } },
  { $sort: { _id: 1 } }
])

db.students.createIndex({ roll_no: 1 }, { unique: true })

db.students.insertOne({ _id: 6, roll_no: "S24CS001", name: "Duplicate Roll" })

db.students.find({ roll_no: "S24AI002" }).explain().queryPlanner.winningPlan

```

8. Sample Input / Database Tables

The following tables of *college_db* (created by Q1-Q3) are used. The values were read from the database after executing the setup queries.

Table: department

dept_id	dept_code	dept_name	building	budget
10	CSE	Computer Engineering	A Block	2500000.00
20	AIML	AI and Machine Learning	B Block	3000000.00
30	ECE	Electronics and Telecomm	C Block	1800000.00
40	MECH	Mechanical Engineering	D Block	1500000.00
50	CIVIL	Civil Engineering	D Block	1200000.00

Table: instructor

instr_id	instr_name	designation	dept_id	salary	email	mentor_id
101	Anil Kulkarni	Professor	10	185000.00	anil.k@example.com	NULL
102	Meera Joshi	Professor	20	172000.00	meera.j@example.com	101
103	Rahul Patil	Asst. Prof	10	82000.00	rahul.p@example.com	101
104	Sneha Deshmukh	Assoc. Prof	20	115000.00	sneha.d@example.com	102
105	Vikram Shinde	Professor	30	168000.00	vikram.s@example.com	101
106	Pooja Kale	Asst. Prof	20	78000.00	pooja.k@example.com	104
107	Nitin Pawar	Asst. Prof	50	76000.00	nitin.p@example.com	105
108	Kavita More	Assoc. Prof	40	112000.00	kavita.m@example.com	105

Table: student

student_id	roll_no	student_name	gender	dob	dept_id	city	email
1	S24CS001	Aarav Sharma	M	2005-03-14	10	Pune	aarav@example.com
2	S24CS002	Isha Gupta	F	2005-07-22	10	Mumbai	isha@example.com
3	S24AI001	Rohan Mehta	M	2005-01-09	20	Pune	rohan@example.com
4	S24AI002	Priya Nair	F	2004-11-30	20	Nashik	priya@example.com
5	S24AI003	Kunal Verma	M	2005-05-18	20	Pune	kunal@example.com
6	S24EC001	Sneha Iyer	F	2005-09-02	30	Nagpur	sneha@example.com
7	S24EC002	Aditya Rao	M	2004-12-25	30	Mumbai	aditya@example.com
8	S24ME001	Neha Kulkarni	F	2005-02-11	40	Pune	neha@example.com
9	S24CS003	Yash Jadhav	M	2005-08-05	10	NULL	yash@example.com
10	S24AI004	Ananya Desai	F	2005-04-27	20	Mumbai	ananya@example.com

Table: course

course_id	title	dept_id	credits	instr_id	prereq_id
AI201	Python for Data Science	20	3	104	CS101
AI301	Machine Learning	20	4	102	AI201
AI401	Deep Learning	20	3	106	AI301
CS101	Programming in C	10	4	103	NULL
CS201	Data Structures	10	4	103	CS101
CS301	Database Management Systems	10	3	101	CS201
CS401	Advanced Databases	10	3	101	CS301
EC201	Digital Electronics	30	3	105	NULL
EC301	Signals and Systems	30	4	NULL	EC201
ME201	Engineering Mechanics	40	3	108	NULL

Table: enrollment (25 rows)

student_id	course_id	semester	marks	grade
1	AI201	3	72	A
1	CS101	3	78	A
1	CS201	3	85	A+
1	CS301	4	91	O
2	CS101	3	88	A+
2	CS201	3	67	B+
2	CS301	4	74	A
3	AI201	3	81	A+
3	AI301	4	89	A+
3	CS101	3	65	B+
3	CS301	4	70	A
4	AI201	3	92	O
4	AI301	4	76	A
4	AI401	4	84	A+
5	AI201	3	58	B
5	AI301	4	63	B+
6	CS101	3	55	B
6	EC201	3	79	A
6	EC301	4	82	A+
7	EC201	3	45	C
7	EC301	4	61	B+
8	CS101	3	78	A
9	CS201	3	90	O
9	CS301	4	95	O
9	CS401	4	88	A+

Table: wallet

student_id	balance
1	60000.00
2	45000.00
3	15000.00
4	80000.00

Table: college_account

acc_id	balance
1	500000.00

The MongoDB documents inserted in Q16 are listed with the query in Section 7.

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine. Q13 and Q14 were executed in separate client sessions logged in as the users created in Q12. The output of Q16 and Q17 is the actual text printed by mongosh 2.x connected to MongoDB 7.0; each command is shown after the mongosh prompt.

Output of Q1: Setup: create college_db and its tables (from Practical 2)

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.00 sec)

mysql> CREATE TABLE department ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE instructor ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE student_phone ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE course ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE enrollment ( ...;
Query OK, 0 rows affected (0.02 sec)
```

Output of Q2: Setup: insert the sample data (from Practical 3)

```
mysql> INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES ...;
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student ...;
Query OK, 10 rows affected (0.01 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_phone VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course VALUES ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES ...;
Query OK, 25 rows affected (0.00 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Output of Q3: Setup: wallet, college account and fee-payment tables

```
mysql> CREATE TABLE wallet (
->     student_id INT PRIMARY KEY,
->     balance     DECIMAL(10,2) NOT NULL,
->     CONSTRAINT chk_balance CHECK (balance >= 0),
->     FOREIGN KEY (student_id) REFERENCES student (student_id)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE college_account (
->     acc_id INT PRIMARY KEY,
->     balance DECIMAL(12,2) NOT NULL
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE fee_payment (
->     receipt_no INT AUTO_INCREMENT PRIMARY KEY,
->     student_id INT NOT NULL,
->     amount     DECIMAL(10,2) NOT NULL CHECK (amount > 0),
->     FOREIGN KEY (student_id) REFERENCES student (student_id)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO wallet VALUES (1, 60000), (2, 45000), (3, 15000), (4, 80000);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO college_account VALUES (1, 500000);
Query OK, 1 row affected (0.00 sec)
```

Output of Q4: Transaction 1: fee payment completed with COMMIT

```
mysql> SELECT @@autocommit;
+-----+
| @@autocommit |
+-----+
|             1 |
+-----+
1 row in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE wallet SET balance = balance - 40000 WHERE student_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> UPDATE college_account SET balance = balance + 40000 WHERE acc_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> INSERT INTO fee_payment (student_id, amount) VALUES (1, 40000);
Query OK, 1 row affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT w.student_id, w.balance AS wallet_balance, a.balance AS college_balance
-> FROM   wallet w, college_account a
-> WHERE  w.student_id = 1 AND a.acc_id = 1;
+-----+-----+-----+
| student_id | wallet_balance | college_balance |
+-----+-----+-----+
|           1 |         20000.00 |         540000.00 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Output of Q5: Transaction 2: changes cancelled with ROLLBACK

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE wallet SET balance = balance - 40000 WHERE student_id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> UPDATE college_account SET balance = balance + 40000 WHERE acc_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT student_id, balance FROM wallet WHERE student_id = 2;
+-----+-----+
| student_id | balance |
+-----+-----+
|          2 | 5000.00 |
+-----+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT student_id, balance FROM wallet WHERE student_id = 2;
+-----+-----+
| student_id | balance |
+-----+-----+
|          2 | 45000.00 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT balance AS college_balance FROM college_account WHERE acc_id = 1;
+-----+
| college_balance |
+-----+
|      540000.00 |
+-----+
1 row in set (0.00 sec)
```

Output of Q6: Transaction 3: partial rollback using SAVEPOINT

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE wallet SET balance = balance - 5000 WHERE student_id = 4;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SAVEPOINT after_exam_fee;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE wallet SET balance = balance - 30000 WHERE student_id = 4;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT student_id, balance FROM wallet WHERE student_id = 4;
+-----+-----+
| student_id | balance |
+-----+-----+
|          4 | 45000.00 |
+-----+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK TO SAVEPOINT after_exam_fee;
Query OK, 0 rows affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT student_id, balance FROM wallet WHERE student_id = 4;
```

```

+-----+-----+
| student_id | balance |
+-----+-----+
|          4 | 75000.00 |
+-----+-----+
1 row in set (0.00 sec)

```

Output of Q7: Transaction 4: an error inside a transaction (atomicity)

```

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE college_account SET balance = balance + 40000 WHERE acc_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE wallet SET balance = balance - 40000 WHERE student_id = 3;
ERROR 3819 (HY000) at line 197: Check constraint 'chk_balance' is violated.

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT (SELECT balance FROM wallet WHERE student_id = 3) AS wallet_3,
-> (SELECT balance FROM college_account WHERE acc_id = 1) AS college_balance;
+-----+-----+
| wallet_3 | college_balance |
+-----+-----+
| 15000.00 | 540000.00 |
+-----+-----+
1 row in set (0.00 sec)

```

Output of Q8: Stored function: grade from marks

```

mysql> CREATE FUNCTION get_grade(p_marks INT)
-> RETURNS CHAR(2)
-> DETERMINISTIC
-> BEGIN
-> ...
-> END$$
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT get_grade(95) AS g95, get_grade(83) AS g83,
-> get_grade(47) AS g47, get_grade(12) AS g12;
+-----+-----+-----+-----+
| g95 | g83 | g47 | g12 |
+-----+-----+-----+-----+
| 0 | A+ | C | F |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT student_id, course_id, marks, grade, get_grade(marks) AS computed
-> FROM enrollment WHERE course_id = 'CS301';
+-----+-----+-----+-----+-----+
| student_id | course_id | marks | grade | computed |
+-----+-----+-----+-----+-----+
| 1 | CS301 | 91 | O | O |
| 2 | CS301 | 74 | A | A |
| 3 | CS301 | 70 | A | A |
| 9 | CS301 | 95 | O | O |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

Output of Q9: Stored procedure with transaction and error handling: pay_fee

```

mysql> CREATE PROCEDURE pay_fee(IN p_student INT, IN p_amount DECIMAL(10,2))
-> BEGIN
-> DECLARE v_balance DECIMAL(10,2);
-> DECLARE EXIT HANDLER FOR SQLEXCEPTION
-> ...
-> END$$

```

```

Query OK, 0 rows affected (0.00 sec)

mysql> CALL pay_fee(2, 40000);
+-----+
| message |
+-----+
| Receipt 2: fee of 40000.00 paid by student 2 |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL pay_fee(3, 40000);
ERROR 1644 (45000) at line 247: Insufficient wallet balance

mysql> SELECT * FROM fee_payment;
+-----+-----+-----+
| receipt_no | student_id | amount |
+-----+-----+-----+
| 1 | 1 | 40000.00 |
| 2 | 2 | 40000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM wallet;
+-----+-----+
| student_id | balance |
+-----+-----+
| 1 | 20000.00 |
| 2 | 5000.00 |
| 3 | 15000.00 |
| 4 | 75000.00 |
+-----+-----+
4 rows in set (0.00 sec)

```

Output of Q10: Stored procedure with OUT parameters

```

mysql> CREATE PROCEDURE dept_stats(IN p_dept_code VARCHAR(10),
->                                OUT p_students INT, OUT p_avg DECIMAL(5,2))
-> BEGIN
->     SELECT COUNT(DISTINCT s.student_id), AVG(e.marks)
->     ...
-> END$$
Query OK, 0 rows affected (0.00 sec)

mysql> CALL dept_stats('AIML', @n, @avg);
Query OK, 1 row affected, 1 warning (0.00 sec)

mysql> SHOW WARNINGS;
+-----+-----+-----+
| Level | Code | Message |
+-----+-----+-----+
| Note | 1265 | Data truncated for column 'p_avg' at row 1 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT @n AS aiml_students, @avg AS aiml_avg_marks;
+-----+-----+
| aiml_students | aiml_avg_marks |
+-----+-----+
| 4 | 75.33 |
+-----+-----+
1 row in set (0.00 sec)

```

Output of Q11: Triggers: automatic grade and audit trail of marks changes

```

mysql> CREATE TABLE marks_audit (
->     audit_id    INT AUTO_INCREMENT PRIMARY KEY,
->     student_id  INT, course_id CHAR(5),
->     old_marks   INT, new_marks INT,
->     changed_by  VARCHAR(60)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TRIGGER trg_enrollment_bi BEFORE INSERT ON enrollment
-> FOR EACH ROW SET NEW.grade = get_grade(NEW.marks);
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TRIGGER trg_enrollment_bu BEFORE UPDATE ON enrollment
-> FOR EACH ROW SET NEW.grade = get_grade(NEW.marks);
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TRIGGER trg_enrollment_au AFTER UPDATE ON enrollment
-> FOR EACH ROW
-> BEGIN
->     IF NOT (OLD.marks <=> NEW.marks) THEN
->         INSERT INTO marks_audit
->             (student_id, course_id, old_marks, new_marks, changed_by)
->             VALUES (OLD.student_id, OLD.course_id, OLD.marks, NEW.marks, USER());
->     END IF;
-> END$$
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks)
-> VALUES (10, 'AI201', 3, 86);
Query OK, 1 row affected (0.00 sec)

mysql> UPDATE enrollment SET marks = 52 WHERE student_id = 7 AND course_id = 'EC201';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM enrollment
-> WHERE (student_id, course_id) IN ((10, 'AI201'), (7, 'EC201'));
+-----+-----+-----+-----+-----+
| student_id | course_id | semester | marks | grade |
+-----+-----+-----+-----+-----+
|          7 | EC201    |         3 |    52 | B     |
|         10 | AI201    |         3 |    86 | A+    |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM marks_audit;
+-----+-----+-----+-----+-----+-----+
| audit_id | student_id | course_id | old_marks | new_marks | changed_by |
+-----+-----+-----+-----+-----+-----+
|         1 |          7 | EC201    |         45 |         52 | root@localhost |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT trigger_name, action_timing, event_manipulation, event_object_table
-> FROM information_schema.triggers
-> WHERE trigger_schema = 'college_db';
+-----+-----+-----+-----+
| TRIGGER_NAME | ACTION_TIMING | EVENT_MANIPULATION | EVENT_OBJECT_TABLE |
+-----+-----+-----+-----+
| trg_enrollment_bi | BEFORE | INSERT | enrollment |
| trg_enrollment_bu | BEFORE | UPDATE | enrollment |
| trg_enrollment_au | AFTER  | UPDATE | enrollment |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

Output of Q12: Role-based access control: roles, privileges and users

```
mysql> DROP USER IF EXISTS 'prof_rahul'@'localhost', 'stu_aarav'@'localhost';
Query OK, 0 rows affected, 2 warnings (0.01 sec)

mysql> DROP ROLE IF EXISTS 'faculty_role', 'student_role';
Query OK, 0 rows affected, 2 warnings (0.00 sec)

mysql> CREATE VIEW v_course_results AS
  -> SELECT s.roll_no, e.course_id, e.marks, e.grade
  -> FROM   enrollment e JOIN student s ON s.student_id = e.student_id;
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE ROLE 'faculty_role', 'student_role';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON college_db.* TO 'faculty_role';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT UPDATE (marks) ON college_db.enrollment TO 'faculty_role';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT EXECUTE ON PROCEDURE college_db.dept_stats TO 'faculty_role';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT ON college_db.v_course_results TO 'student_role';
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE USER 'prof_rahul'@'localhost' IDENTIFIED BY 'Faculty@123';
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE USER 'stu_aarav'@'localhost' IDENTIFIED BY 'Student@123';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT 'faculty_role' TO 'prof_rahul'@'localhost';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT 'student_role' TO 'stu_aarav'@'localhost';
Query OK, 0 rows affected (0.00 sec)

mysql> SET DEFAULT ROLE ALL TO 'prof_rahul'@'localhost', 'stu_aarav'@'localhost';
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW GRANTS FOR 'faculty_role';
+-----+
| Grants for faculty_role@%          |
+-----+
| GRANT USAGE ON *.* TO `faculty_role`@`%` |
| GRANT SELECT ON `college_db`.* TO `faculty_role`@`%` |
| GRANT UPDATE (`marks`) ON `college_db`.`enrollment` TO `faculty_role`@`%` |
| GRANT EXECUTE ON PROCEDURE `college_db`.`dept_stats` TO `faculty_role`@`%` |
+-----+
4 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'stu_aarav'@'localhost' USING 'student_role';
+-----+
| Grants for stu_aarav@localhost      |
+-----+
| GRANT USAGE ON *.* TO `stu_aarav`@`localhost` |
| GRANT SELECT ON `college_db`.`v_course_results` TO `stu_aarav`@`localhost` |
| GRANT `student_role`@`%` TO `stu_aarav`@`localhost` |
+-----+
3 rows in set (0.00 sec)
```

Output of Q13: Testing the faculty account

Session user: prof_rahul@localhost

```
mysql> SELECT CURRENT_USER(), CURRENT_ROLE();
+-----+
| CURRENT_USER()          | CURRENT_ROLE()          |
```

```

+-----+-----+-----+-----+
| prof_rahul@localhost | `faculty_role`@`%` |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM v_course_results WHERE course_id = 'CS101';
+-----+-----+-----+-----+
| roll_no | course_id | marks | grade |
+-----+-----+-----+-----+
| S24CS001 | CS101     | 78    | A      |
| S24CS002 | CS101     | 88    | A+     |
| S24AI001 | CS101     | 65    | B+     |
| S24EC001 | CS101     | 55    | B      |
| S24ME001 | CS101     | 78    | A      |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql> UPDATE enrollment SET marks = 80 WHERE student_id = 8 AND course_id = 'CS101';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE enrollment SET grade = 'O' WHERE student_id = 8 AND course_id = 'CS101';
ERROR 1143 (42000) at line 5: UPDATE command denied to user 'prof_rahul'@'localhost' for
column 'grade' in table 'enrollment'

mysql> DELETE FROM student WHERE student_id = 8;
ERROR 1142 (42000) at line 6: DELETE command denied to user 'prof_rahul'@'localhost' for
table 'student'

mysql> CALL dept_stats('CSE', @n, @avg);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @n, @avg;
+-----+-----+
| @n   | @avg  |
+-----+-----+
| 3    | 82.80 |
+-----+-----+
1 row in set (0.00 sec)

```

Output of Q14: Testing the student account

Session user: stu_aarav@localhost

```

mysql> SELECT * FROM v_course_results WHERE roll_no = 'S24CS001';
+-----+-----+-----+-----+
| roll_no | course_id | marks | grade |
+-----+-----+-----+-----+
| S24CS001 | AI201     | 72    | A      |
| S24CS001 | CS101     | 78    | A      |
| S24CS001 | CS201     | 85    | A+     |
| S24CS001 | CS301     | 91    | O      |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM wallet;
ERROR 1142 (42000) at line 3: SELECT command denied to user 'stu_aarav'@'localhost' for table
'wallet'

mysql> UPDATE v_course_results SET marks = 100 WHERE roll_no = 'S24CS001';
ERROR 1142 (42000) at line 4: UPDATE command denied to user 'stu_aarav'@'localhost' for table
'v_course_results'

```

Output of Q15: Audit trail after the faculty update; revoking a privilege

```
mysql> SELECT * FROM marks_audit;
+-----+-----+-----+-----+-----+-----+
| audit_id | student_id | course_id | old_marks | new_marks | changed_by |
+-----+-----+-----+-----+-----+-----+
| 1 | 7 | EC201 | 45 | 52 | root@localhost |
| 2 | 8 | CS101 | 78 | 80 | prof_rahul@localhost |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> REVOKE UPDATE (marks) ON college_db.enrollment FROM 'faculty_role';
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW GRANTS FOR 'faculty_role';
+-----+-----+-----+-----+-----+-----+
| Grants for faculty_role@% |
+-----+-----+-----+-----+-----+-----+
| GRANT USAGE ON *.* TO `faculty_role`@`%` |
| GRANT SELECT ON `college_db`.* TO `faculty_role`@`%` |
| GRANT EXECUTE ON PROCEDURE `college_db`.`dept_stats` TO `faculty_role`@`%` |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Output of Q16: Basic NoSQL operations in MongoDB (mongosh): create, read, update, delete

```
test> use college_nosql
switched to db college_nosql

college_nosql> db.students.drop()
true

college_nosql> db.students.insertMany([
...   { _id: 1, roll_no: "S24CS001", name: "Aarav Sharma", dept: "CSE", city: "Pune",
...     skills: ["C", "SQL"],
...     courses: [ { code: "CS101", marks: 78 }, { code: "CS301", marks: 91 } ] },
...   { _id: 2, roll_no: "S24CS002", name: "Isha Gupta", dept: "CSE", city: "Mumbai",
...     skills: ["Java"],
...     courses: [ { code: "CS101", marks: 88 }, { code: "CS301", marks: 74 } ] },
...   { _id: 3, roll_no: "S24AI001", name: "Rohan Mehta", dept: "AIML", city: "Pune",
...     skills: ["Python"],
...     courses: [ { code: "AI201", marks: 81 }, { code: "AI301", marks: 89 } ] },
...   { _id: 4, roll_no: "S24AI002", name: "Priya Nair", dept: "AIML", city: "Nashik",
...     skills: ["Python", "R"],
...     courses: [ { code: "AI201", marks: 92 }, { code: "AI401", marks: 84 } ] },
...   { _id: 5, roll_no: "S24EC001", name: "Sneha Iyer", dept: "ECE", city: "Nagpur",
...     courses: [ { code: "EC201", marks: 79 } ] }
... ])
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
}

college_nosql> db.students.find({ city: "Pune" }, { _id: 0, roll_no: 1, name: 1, dept: 1 })
[
  { roll_no: 'S24CS001', name: 'Aarav Sharma', dept: 'CSE' },
  { roll_no: 'S24AI001', name: 'Rohan Mehta', dept: 'AIML' }
]

college_nosql> db.students.find({ "courses.marks": { $gte: 90 } }, { _id: 0, name: 1 })
[ { name: 'Aarav Sharma' }, { name: 'Priya Nair' } ]

college_nosql> db.students.countDocuments({ dept: "AIML" })
2

college_nosql> db.students.updateOne({ _id: 3 },
...   { $set: { city: "Mumbai" }, $push: { skills: "TensorFlow" } })
{
  acknowledged: true,
  insertedId: null,
```

```

    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }

college_nosql> db.students.updateMany({ dept: "AIML" }, { $set: { year: "SY" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}

college_nosql> db.students.findOne({ _id: 3 })
{
  _id: 3,
  roll_no: 'S24AI001',
  name: 'Rohan Mehta',
  dept: 'AIML',
  city: 'Mumbai',
  skills: [ 'Python', 'TensorFlow' ],
  courses: [ { code: 'AI201', marks: 81 }, { code: 'AI301', marks: 89 } ],
  year: 'SY'
}

college_nosql> db.students.deleteOne({ _id: 5 })
{ acknowledged: true, deletedCount: 1 }

college_nosql> db.students.countDocuments()
4

```

Output of Q17: MongoDB aggregation pipeline, unique index and query plan

```

test> use college_nosql
switched to db college_nosql

college_nosql> db.students.aggregate([
...   { $unwind: "$courses" },
...   { $group: { _id: "$dept", enrollments: { $sum: 1 },
...             avg_marks: { $avg: "$courses.marks" } } },
...   { $sort: { _id: 1 } }
... ])
[
  { _id: 'AIML', enrollments: 4, avg_marks: 86.5 },
  { _id: 'CSE', enrollments: 4, avg_marks: 82.75 }
]

college_nosql> db.students.createIndex({ roll_no: 1 }, { unique: true })
roll_no_1

college_nosql> db.students.insertOne({ _id: 6, roll_no: "S24CS001", name: "Duplicate Roll" })
Uncaught
MongoServerError: E11000 duplicate key error collection: college_nosql.students index:
  roll_no_1 dup key: { roll_no: "S24CS001" }

college_nosql> db.students.find({ roll_no: "S24AI002" }).explain().queryPlanner.winningPlan
{
  stage: 'FETCH',
  inputStage: {
    stage: 'IXSCAN',
    keyPattern: { roll_no: 1 },
    indexName: 'roll_no_1',
    isMultiKey: false,
    multiKeyPaths: { roll_no: [ ] },
    isUnique: true,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: { roll_no: [ '['S24AI002', 'S24AI002']' ] }
  }
}

```

}

10. Result

Transactions behaved atomically: the committed payment changed both balances, the rolled-back payment restored the original values, ROLLBACK TO SAVEPOINT undid only the second deduction, and the CHECK violation inside a transaction was undone completely by ROLLBACK. The function get_grade, the procedures pay_fee (with SIGNAL/RESIGNAL) and dept_stats (OUT parameters) and the three triggers worked as designed, including the audit record of the faculty user's change. RBAC allowed the faculty user to read data and update only marks, and the student user to read only the results view; all other operations were denied with ERROR 1142/1143. CRUD, aggregation and unique-index operations were performed successfully in MongoDB.

11. Conclusion

Transactions protect the database from partial updates; together with constraints they provide the ACID guarantees needed for financial operations such as fee payment. Stored functions, procedures and triggers move business rules into the database, where they are applied consistently for every application and user. Roles make security administration simple and enforce the principle of least privilege, especially when combined with views and column-level privileges. MongoDB showed that a document model can store related data (student with courses) in one flexible document, which suits evolving schemas, whereas relational databases remain preferable when strong consistency across many related tables is required.